Lista Enlazada Simple

Nombre del Auxiliar

Tiempo estimado: 60 min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Primer Semestre 2027
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

1.1 Valores

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Responsabilidad Técnica y Precisión en el Manejo de Recursos	El estudiante evidencia responsabilidad al verificar meticulosamente el orden de asignación de las referencias. Comprende que un error en la lógica de punteros (como romper un enlace antes de crear el nuevo) resulta en la pérdida irreversible de acceso a los datos en el Heap.

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Implementa estructuras de datos lineales y no lineales usando programación orientada a objetos para desarrollar soluciones eficientes
Competencia Específica	Construye y gestiona estructuras de datos no contiguas mediante la implementación manual de listas enlazadas en Perl, utilizando referencias de autorreferencia y el mecanismo de blessing para la creación de objetos, lo que le permite manipular la memoria dinámica del sistema y garantizar la conectividad lógica de la información sin depender de estructuras de tamaño fijo

2. Palabras Clave

- **Nodo:** La unidad básica de una lista enlazada que contiene tanto el valor del dato como uno o más punteros hacia otros nodos.
- **Puntero de Autorreferencia:** Un campo dentro de una estructura o clase que apunta a otra instancia del mismo tipo, permitiendo la creación de cadenas de datos.
- **Gestión Dinámica de Memoria:** El proceso de asignar y liberar espacio en el *Heap* durante la ejecución del programa, permitiendo que la estructura crezca o se encoja según sea necesario.
- **Encabezado (Head):** Puntero especial que marca el inicio de la lista y sirve como punto de entrada único para realizar cualquier operación de recorrido.
- **Asignación No Contigua:** Característica de las listas donde los elementos se almacenan en direcciones de memoria dispersas, a diferencia de los arreglos donde todos los elementos están uno tras otro.
- **Desreferenciación:** El acto de acceder al valor almacenado en la dirección de memoria a la que apunta un puntero, esencial para navegar entre los nodos de la lista.

- **Recorrido Secuencial:** Método de acceso donde, para llegar al elemento *n*, es obligatorio pasar por todos los nodos anteriores, dado que la ubicación del siguiente nodo solo se conoce estando en el actual.
- **Lista Doblemente Enlazada:** Variante de la estructura donde cada nodo posee dos punteros siguiente y anterior, permitiendo la navegación bidireccional y una mayor flexibilidad en la eliminación de nodos.

3. Resumen

Este ejercicio demostrativo presenta de forma guiada la implementación manual de estructuras de datos no contiguas, centrándose en la Lista Enlazada en sus variantes simple y doble, así como en la gestión dinámica de nodos en memoria. El ejercicio corresponde a los contenidos de la Semana 3 del curso Estructuras de Datos [B].

La clase se orienta a que el estudiante comprenda la naturaleza de la asignación dinámica de memoria, identifique la diferencia entre el acceso aleatorio de los arreglos y el recorrido secuencial mediante punteros, e implemente correctamente la lógica de autorreferencia para la inserción, eliminación y búsqueda de elementos. El objetivo es que el estudiante aprenda a manipular direcciones de memoria de forma manual, asegurando la integridad de los enlaces y la liberación responsable de recursos en el *heap* sin depender de librerías externas.

4.Contenido

Descripción del ejercicio

El ejercicio demostrativo consiste en implementar desde cero, sin utilizar librerías estándar, la estructura de una Lista Enlazada Simple y una Lista Doblemente Enlazada, fundamentadas en la gestión dinámica de memoria. A diferencia de las estructuras estáticas, el ejercicio se centra en la creación de nodos y la manipulación de punteros de autorreferencia para conectar elementos dispersos.

Objetivo de la demostración

Demostrar el procedimiento para implementar manualmente las variantes de Listas Enlazadas mediante nodos en Java, aplicando operaciones de inserción (inicio, final, posición n), recorrido secuencial y eliminación con liberación lógica de recursos. El fin es que el estudiante comprenda la naturaleza de la memoria no contigua, identifique el rol de los punteros head, tail, next y prev, y desarrolle la capacidad técnica para mantener la integridad de una estructura dinámica durante su manipulación individual.

Recursos necesarios

Software y entorno

- **Intérprete:** Perl 5.10 o superior (disponible por defecto en la mayoría de sistemas Linux/macOS o vía Strawberry Perl en Windows).
- **IDE:** Visual Studio Code con las extensiones **"Perl Navigator"** (para autocompletado y sintaxis) y **"Perl"** de Gerald Richter.
- **Terminal:** Integrada en VS Code para la ejecución de scripts (perl script.pl).

Herramientas técnicas

- **Debugger:** Uso de perl -d para inspeccionar la jerarquía de referencias en tiempo real.
- **Data::Dumper:** Módulo núcleo de Perl indispensable para visualizar la estructura de los nodos y sus enlaces (print Dumper($lista)).
- **Pizarrón o pantalla compartida:** Crucial para diagramar cómo las referencias de Perl apuntan a diferentes direcciones de memoria.

Materiales de apoyo

- **Diapositivas de la Semana 3:** Actualizadas con definiciones de Listas Enlazadas Simples y Dobles y el concepto de Referencias en Perl.
- **Archivo base Lista.pm:** Módulo de Perl con la definición del paquete Nodo y los métodos base (new, insertar, eliminar) ya estructurados.

- **Guía de Sintaxis:** Hoja de referencia sobre el uso de la flecha (->) para desreferenciar y acceder a campos de un Hash como si fueran atributos de un objeto.

Configuración mínima

- **Sistema:** Perl 5.20+ (recomendado para mejores funciones de firmas de subrutinas).
- **RAM:** 4 GB RAM (Perl es muy ligero, por lo que corre fluidamente en VS Code).
- **Monitor:** Proyector para demostrar la ejecución en terminal y el volcado de datos con Dumper.

Desarrollo paso a paso

Nota: El procedimiento está redactado para que cualquier tutor pueda reproducirlo en semestres futuros. Cada paso incluye qué hacer, por qué hacerlo y qué observar.

Paso 1 - Introducción conceptual (10 min)

Presentar las diapositivas de la Semana 5 enfocadas en la Gestión Dinámica de Memoria. Dibujar en el pizarrón la diferencia visual entre un Arreglo Memoria Contigua y una Lista Enlazada Memoria Dispersa: un bloque sólido de casillas numeradas frente a una serie de "vagonetas" conectadas por cadenas punteros/referencias.

Por qué hacerlo: El estudiante debe comprender que, a diferencia de las estructuras estáticas, en las listas la ubicación física del dato es irrelevante; lo que importa es la referencia que lo conecta al siguiente. Es fundamental introducir aquí el concepto de Referencia en Perl (el uso del operador \ y la flecha ->), ya que es el mecanismo que sustituye a los punteros tradicionales de lenguajes como C++ o Java.

Paso 2 — Definición de la estructura y el "blessing" (15 min)

Qué hacer: Explicar cómo Perl simula la orientación a objetos para crear nodos. En el código, la función bless toma una referencia a un hash (que contiene valor y sig) y la convierte en un objeto de la clase Nodo. Se debe enfatizar que el campo sig inicializado en undef es el equivalente al null de otros lenguajes.

Decisiones técnicas y validaciones:

- **Encapsulamiento manual:** Al usar package Nodo, estamos separando la lógica del contenedor de la lógica del dato.
- **Referencias:** Es vital explicar que $self->{cabeza} no guarda el dato, sino la dirección de memoria donde inicia la cadena. Sin esta dirección, la lista se pierde para el recolector de basura de Perl.

lista_simple.pl U

lista_simple.pl

```perl
#!/usr/bin/perl
use strict;
use warnings;

# --- Clase Nodo ---
package Nodo;
sub nuevo {
    my ($clase, $valor) = @_;
    return bless {
        valor => $valor,
        sig   => undef,
    }, $clase;
}

# --- Clase Lista Simple ---
package ListaSimple;
sub nueva {
    my ($clase) = @_;
    return bless { cabeza => undef }, $clase;
}
```

Paso 3 — Implementación de la lógica de inserción al final (20 min)
Qué hacer:

Describir el algoritmo de inserción en la clase ListaSimple. El tutor debe mostrar dos escenarios en la pizarra antes de codificar:

1. **Lista vacía:** La cabeza apunta directamente al nuevo_nodo.
2. **Lista con datos:** Se utiliza un puntero auxiliar ($actual) para realizar un recorrido secuencial mediante un ciclo while hasta encontrar el último nodo (aquel cuyo sig es undef).

Interpretación de resultados:

Antes de continuar, el estudiante debe notar que la complejidad de esta inserción es O(n), ya que conforme la lista crece, el programa tarda más en encontrar el final. Esto ayuda a verificar que los enlaces se están creando correctamente sin sobrescribir los datos existentes.

```perl
sub insertar {
    my ($self, $valor) = @_;
    my $nuevo_nodo = Nodo->nuevo($valor);
    if (!$self->{cabeza}) {
        $self->{cabeza} = $nuevo_nodo;
    } else {
        my $actual = $self->{cabeza};
        while ($actual->{sig}) { $actual = $actual->{sig}; }
        $actual->{sig} = $nuevo_nodo;
    }
}
```

Paso 4 — Cierre técnico y visualización del volcado (15 min)

Qué hacer: Ejecutar el método mostrar y explicar la importancia de la condición while ($actual). Al llegar al final de la lista, el puntero se vuelve undef, lo cual rompe el ciclo de impresión.

Confirmación de resolución: Para confirmar que el ejercicio es correcto, se recomienda al estudiante:

1. Verificar que la salida termine en NULL.
2. **Opcional pero recomendado:** Usar el módulo Data::Dumper para ver la estructura real en memoria. Si el estudiante ve una serie de hashes anidados dentro de otros, ha logrado implementar la lista con éxito.
    - Comando de verificación: use Data::Dumper; print Dumper($lista);

```perl
sub mostrar {
    my ($self) = @_;
    my $actual = $self->{cabeza};
    while ($actual) {
        print "[" . $actual->{valor} . "] -> ";
        $actual = $actual->{sig};
    }
    print "NULL\n";
}
```

```perl
# --- Programa Principal ---
package main;
my $lista = ListaSimple->nueva();
$lista->insertar("Dato 1");
$lista->insertar("Dato 2");
$lista->insertar("Dato 3");
print "Estado de la lista:\n";
$lista->mostrar();
```

Tabla 1 - Tabla de Análisis de la Estructura y Flujo de Datos

Elemento	Descripción
Entrada de Datos	Cadenas de texto (Scalars en Perl) enviadas al método insertar, por ejemplo: "Dato 1", "Dato 2".
Puntero de Referencia	El hash key {sig}, que almacena la dirección de memoria del siguiente nodo o el valor especial undef.
Mecánica de Recorrido	Uso de una variable auxiliar $actual que se desplaza de forma secuencial saltando de referencia en referencia ($actual = $actual->{sig}).

Resultado esperado

Al finalizar la ejecución del script en la terminal de **Visual Studio Code**, el producto final debe ser una representación lineal y fiel de la secuencia de datos ingresados. La evidencia de éxito se manifiesta en dos niveles:

1. **Salida Visual:** La consola debe mostrar la cadena de nodos conectada por flechas, iniciando con el primer dato insertado y finalizando estrictamente con la palabra NULL (ej. [Dato 1] -> [Dato 2] -> [Dato 3] -> NULL).
2. **Comportamiento Lógico:** El programa no debe presentar errores de segmentación o referencias no definidas (Can't use string as a HASH ref...). Si se añade un cuarto dato en el código principal, este debe aparecer automáticamente al final de la cadena tras una nueva ejecución, confirmando que la lógica de recorrido secuencial y asignación dinámica funciona correctamente.

Aplicación práctica

Este ejercicio trasciende la implementación de una lista simple; es la base fundamental para entender cómo operan los sistemas modernos en niveles de abstracción bajos y altos:

- **En el curso:** Es el requisito previo para construir estructuras más complejas como Grafos o Árboles B esenciales para el manejo de índices en bases de datos.
- **En otras prácticas:** La lógica de punteros y referencias en Perl te permitirá gestionar de manera eficiente la migración de datos entre diferentes capas de una arquitectura, donde los datos suelen estructurarse como objetos dinámicos.
- **En el ámbito profesional:** El manejo de memoria dinámica es clave en el desarrollo de Sistemas Operativos, Motores de Videojuegos y Sistemas de Telecomunicaciones, donde el tamaño de la información es volátil y no puede limitarse a arreglos de tamaño fijo. Dominar esto te permite escribir código que no solo funciona, sino que optimiza el uso del hardware.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

Dificultades encontradas

- **Abstracción de Referencias:** A diferencia de lenguajes con punteros explícitos, en Perl el concepto de "referencia a un hash" puede ser confuso. Los estudiantes suelen olvidar que están manipulando la dirección de memoria y no el contenido del hash directamente.
- **Sintaxis de Flecha (->):** Existe una tendencia inicial a intentar acceder a los datos mediante claves de hash estándar ($nodo{valor}), lo cual genera errores de ejecución. Es vital recalcar el uso de la flecha para desreferenciar.
- **Lógica de Final de Lista:** El manejo de undef como condición de parada frecuentemente causa errores de "uso de valor no definido" si no se valida correctamente antes de acceder al siguiente enlace.

Recomendaciones para futuras sesiones

- **Uso de Visualizadores:** Utilizar el módulo Data::Dumper desde el inicio para que el estudiante "vea" cómo se anidan los nodos. La visualización física de la memoria ayuda a consolidar el concepto de enlace.
- **Diagramación Previa:** No iniciar el código en VS Code sin haber dibujado antes el movimiento de las referencias en el pizarrón. El código debe ser la traducción de un dibujo previo.
- **Comparativa Constante:** Reforzar constantemente por qué esta lista es más flexible que un arreglo de Perl (@array), mencionando la eficiencia en inserciones en el inicio o medio de la estructura.

6. Referencias

1. **Wall, L., Christiansen, T., & Orwant, J. (2012).** *Programming Perl: Unmatched power for text processing and scripting* (4th ed.). O'Reilly Media. https://www.oreilly.com/library/view/programming-perl-4th/9781449321451/
2. **Conway, D. (2005).** *Perl Best Practices*. O'Reilly Media.. https://www.oreilly.com/library/view/perl-best-practices/0596001738/